Postmortem: RADIUS / VPN-only voucher authentication

Summary

After migrating the platform to a VPN-only topology (routers reachable only over WireGuard), voucher authentication was completely broken on every router: clients entered a valid voucher code and the captive portal silently reloaded the login page. Diagnosis and repair spanned a long debugging session and uncovered roughly a dozen distinct bugs. Almost all of them shared a small number of root-cause *classes* rather than being independent — the same underlying mismatch surfaced repeatedly in different subsystems. By the end, the full voucher lifecycle (auth → accounting → cap enforcement → CoA disconnect → backstop recovery) was proven working end-to-end on multiple routers.

The central lesson: the development environment used direct router IPs and a permissive catch-all RADIUS client, so it never exercised the VPN-only paths. "Works in dev" did not predict "works in prod" for anything touching RADIUS, CoA, routing, or source-IP identity. Every bug below was invisible in dev and only surfaced under real production topology.

What was broken (by failure class, not individual bug)

Class 1 — `ip_address` vs `wg_tunnel_ip` (the dominant class)

In a VPN-only topology a router's `ip_address` column is a display-only label; all real connectivity uses `wg_tunnel_ip`. Multiple independent code paths were still using `ip_address` (or a configured public host) where they needed the tunnel IP. Each manifested as a different, separately-diagnosed symptom:

- RADIUS server address pushed to routers used the public host, not the tunnel — routers reported "Network unreachable," no packet ever reached FreeRADIUS.
- The SQL `client_query` keyed NAS clients on `ip_address` — tunneled routers either had NULL `ip_address` or a source IP that didn't match, so no per-router client loaded.
- The authorize/accounting SQL joined `routers` on `NAS-Identifier` (the router's editable system identity) instead of the tunnel IP — auth reached FreeRADIUS, matched the client, then returned zero rows because the identity didn't match the DB value.
- CoA/Disconnect targeted `ip_address` (NULL/fake for tunneled routers) — disconnects could never reach the router.
- The accounting heartbeat (`_mark_router_heartbeat`) matched `ip_address == nas_ip`, but accounting packets carry the tunnel IP as NAS-IP — so VPN routers never registered as online; all routers showed `is_online = false` with stale `last_seen_at`.

Why invisible until prod: dev used direct IPs, so `ip_address` *was* the reachable address and *did* match the source IP. The tunnel-vs-label distinction simply didn't exist in dev.

What would have caught it: any environment where a router is reachable *only* via tunnel, so a code path using `ip_address` fails loudly instead of coincidentally working.

Class 2 — the routeros_api `.id` vs `id` key

The `routeros_api` library returns an entry's ID under the key `id` (no dot) for most paths, but code throughout the RouterOS-facing layer read `.get(".id")`, which always returned `None`. This silently broke ~9 sites: every "find existing entry then set-or-add" upsert fell through to "add" (creating duplicates), and every "find then bind/skip" fell through to skip. Symptoms looked unrelated:

- `/ip/hotspot/profile/set` failed with "missing .id".
- Re-applying RADIUS config created duplicate `/radius` entries (a stale unreachable one ordered first, poisoning auth).
- The hotspot server was never bound to the configured profile (`hsprof1`), so it ran on the default profile with `use-radius=false` — RADIUS was never invoked at all. This hit every router the wizard provisioned.
- The disconnect button sent `active_id: null`, rejected by request validation before any router call.

Why invisible until prod: all of these are RouterOS-interaction paths that the dev flow either didn't exercise against a real router or didn't re-run idempotently.

What would have caught it: integration tests (or a staging run) that exercise the provisioning wizard against a real RouterOS instance and assert the resulting router state (profile bound, single RADIUS entry, RADIUS enabled), plus re-running apply twice to catch non-idempotent upserts.

Class 3 — environment/topology-specific config that only breaks under prod networking

- `sql.conf` hardcoded `host=postgres` (Docker DNS). Under `network_mode: host` (required so FreeRADIUS sees true tunnel source IPs) there is no Docker DNS, so the SQL module couldn't reach the database — zero clients, indistinguishable from a broken query.
- `sql.conf` hardcoded the wrong DB password (a dev placeholder), so even after the host was resolved the connection failed auth.
- `client_query` returned the wrong column count/order for FreeRADIUS 3.2.x ("too few fields"), so even matched rows were rejected.

- Docker `userland-proxy=true` rewrote RADIUS source IPs to the bridge gateway under bridge networking, masking per-router identity — which is *why* host-networking was needed, but also forced HA/bind-address rework.

Why invisible until prod: dev ran bridge-networked with Docker DNS and the dev DB credentials; the host-networking move was prod-only.

What would have caught it: a staging environment that runs FreeRADIUS host-networked exactly as prod does, against a real Postgres, started the same way.

Class 4 — silent success masking failure (a cross-cutting amplifier)

Several bugs were hard to find not because they were subtle but because the failing operation *reported success*:

- The admin `apiCall` returned non-2xx response bodies as if they were data (only threw on 401), so a 404 on credential-save looked like success while nothing was written.
- A wizard step that didn't actually bind the profile returned success.
- RADIUS auth succeeded while accounting silently failed (no interim updates), so usage froze while the session looked healthy.
- CoA could fail entirely while the voucher was marked exhausted — the DB showed a clean state while the client stayed online for days (the gap the backstop now closes).

Why dangerous: silent success means the symptom appears far from the cause, and "it worked" in one layer hides a failure in the next.

What would have caught it: stricter error propagation (the `apiCall` audit fixed this class in the frontend), read-back verification after state-changing operations (the wizard now reads back the profile binding), and end-to-end tests that assert the *downstream* effect (session usage actually climbs), not just the immediate response.

What went well

- Diagnosis discipline: each bug was traced to a confirmed root cause via live `freeradius -X` traces and direct DB/router inspection before fixing — not patched speculatively.
- Fixing classes, not instances: the `.id` bug was resolved by routing every site through a shared helper and grepping the whole codebase for stragglers, rather than fixing each symptom. Same for the `apiCall` error-handling audit.
- Report-before-apply on anything touching live networking, secrets, or the DB — which repeatedly surfaced a better fix than the first instinct (e.g. host-networking revealed the `host=postgres` blocker before it shipped).

- Design-before-code on the genuinely-optional-shaped decisions (enforcement granularity, the exhausted-session backstop), which caught load-bearing design errors (the backstop's original detection gate would have missed the successful-CoA-then-lost-Acct-Stop case).

Structural fixes adopted

- All RouterOS ID access routes through `_routeros_id()` / `_row_id()`, with a comment explaining the library quirk so it isn't reintroduced.
- `apiCall` throws on any non-2xx; call sites surface errors instead of proceeding on bad data.
- RADIUS/CoA/accounting paths resolve the router by `wg_tunnel_ip` (the tunnel-IP / source-IP key), not `ip_address` or `NAS-Identifier`.
- Dynamic FreeRADIUS clients (SQL lookup on first packet) replaced restart-on-router-change — new routers authenticate with zero restart and no cross-tenant disruption; secret rotation and deactivation apply within the cache TTL.
- A backstop job recovers exhausted-but-still-connected sessions via the RouterOS API path (independent of CoA), closing the "CoA failed, client rides free" leak.

The generating condition (the real takeaway)

Most of this session's work traces to a single structural gap: **the development environment did not reproduce production topology.** Dev used direct router IPs and a permissive catch-all RADIUS client; prod is VPN-only with per-router secrets and host-networked FreeRADIUS. Every Class 1–3 bug "worked" in dev for a reason that didn't hold in prod. The fixes above close the specific instances; they do not close the gap that generated them.

The structural mitigation is a **production-faithful staging environment** that exercises: a real WireGuard tunnel (true tunnel source IPs), host-networked FreeRADIUS against a real Postgres started as prod starts it, per-router secrets with dynamic client loading, and at least one router reachable *only* via tunnel so any residual `ip_address` dependency fails loudly. The test of that staging design is this very postmortem: for each failure class above, "would staging have caught it?" must be yes.

Without that, the next change that touches RADIUS, CoA, routing, or source-IP identity is liable to reproduce this class of "passes in dev, fails in prod" surprise — and once real operators are onboarded, that failure lands on their paying customers, live.